

MÓDULO 10

CI/CD, Control de Versiones y Operación en Equipos

codex exec · GitHub Action · Auto-fix · Cloud · Worktrees · Team Config

Plan de Formación — OpenAI Codex para Desarrolladores

Nivel: Avanzado · Duración estimada: 3–4 horas

Versión 1.0 — Febrero 2026

Índice

Índice.....	2
1. Introducción.....	4
2. Objetivos de aprendizaje	4
3. Contenidos teóricos	5
3.1 codex exec — Modo no interactivo.....	5
3.2 GitHub Action (openai/codex-action@v1).....	5
3.3 Auto-fix de CI fallido	6
3.4 codex exec resume — Pipelines multi-paso	6
3.5 Codex Cloud.....	6
3.6 Worktrees en la App.....	7
3.7 Buenas prácticas Git con Codex (oficial).....	7
3.8 Configuración para equipos	7
3.8.1 requirements.toml (Enterprise).....	7
3.8.2 Perfiles compartidos	7
3.8.3 Execution policy rules	7
3.8.4 OTel para auditoría.....	8
4. Buenas prácticas	9
4.1 Codex propone, humano aprueba	9
4.2 PRs pequeños.....	9
4.3 CI como árbitro.....	9
4.4 Commits convencionales	9
4.5 Execution rules para proteger	9
4.6 Límites de auto-fix	9
4.7 Secrets solo en setup.....	9
5. Errores comunes.....	10
5.1 Reescribir historial sin policy	10
5.2 Automatizar merges sin revisión.....	10
5.3 danger-full-access en CI compartido	10
5.4 No pinear versión de Codex.....	10
5.5 Prompts de CI sin constraints	10
6. Casos de uso reales	11
6.1 Migración gradual.....	11
6.2 Release notes automáticas.....	11
6.3 Changelogs estructurados	11
6.4 Triage de issues	11
7. Laboratorio práctico (L10) — PR Listo para Producción.....	12

7.1 Paso 1 — Crear proyecto con CI simulado	12
7.2 Paso 2 — AGENTS.md y config de equipo	12
7.3 Paso 3 — Feature branch + cambios con Codex	12
7.4 Paso 4 — PR description con codex exec	12
7.5 Paso 5 — /review antes de merge	13
7.6 Paso 6 — Simular auto-fix con codex exec	13
7.7 Verificación final	13
7.8 Limpieza (Protocolo Obligatorio)	13
8. Resumen — Cierre del Plan de Formación	15
9. Referencias	15

1. Introducción

Este módulo cierra el plan de formación integrando todo lo aprendido en flujos profesionales de CI/CD y trabajo en equipo. Codex no es solo una herramienta de desarrollo individual — es una pieza operable en pipelines de integración continua, revisión de PRs y automatización de tareas repetibles. Aprenderemos a usar codex exec para scripts y CI, la GitHub Action oficial, auto-fix de tests fallidos, Codex Cloud para tareas delegadas, y configuración de equipo con requirements.toml, perfiles compartidos y execution policy rules.

2. Objetivos de aprendizaje

#	Objetivo	Evidencia de logro
O1	Integrar Codex en CI/CD con codex exec y GitHub Action.	Pipeline local con auto-fix funcionando.
O2	Estandarizar flujos Git: branches, commits, PRs, /review.	PR con commits convencionales y checklist.
O3	Configurar Codex para equipos: requirements.toml, reglas, OTel.	Config compartida aplicada al proyecto.

3. Contenidos teóricos

3.1 codex exec — Modo no interactivo

codex exec ejecuta Codex sin el TUI, diseñado para scripts y CI/CD. El progreso se envía a stderr; el mensaje final a stdout.

Flag	Descripción
(sin flags)	Sandbox read-only, output a stdout
--full-auto	workspace-write + aprobación automática
--json	Output JSON Lines para consumo programático
-o <path>	Guardar mensaje final en fichero
--output-schema	Respuesta final con JSON Schema
--ephemeral	No persiste ficheros de sesión
--skip-git-repo-check	Permitir ejecución fuera de repo Git

```
# Release notes
codex exec "generate release notes for the last 10 commits" \
  | tee release-notes.md

# JSON Lines
codex exec --json "summarize the repo" | jq

# Output schema
codex exec "Extract metadata" \
  --output-schema ./schema.json -o ./metadata.json

# Ephemeral
codex exec --ephemeral "triage this repository"
```

3.2 GitHub Action (openai/codex-action@v1)

La GitHub Action oficial instala Codex CLI, arranca el proxy de la API y ejecuta codex exec con los permisos especificados.

Input	Descripción
openai-api-key	Secret con API key
prompt / prompt-file	Prompt inline o fichero .md
sandbox	read-only, workspace-write, danger-full-access
safety-strategy	drop-sudo (default), unprivileged-user, unsafe
output-file	Guardar mensaje final
output-schema-file	JSON Schema para output estructurado
codex-version	Pinear versión específica
model	Modelo a usar

⚠ safety-strategy: drop-sudo

Elimina sudo antes de ejecutar Codex. Irreversible para el job. Protege secrets en memoria. Es el default y la opción recomendada para CI.

3.3 Auto-fix de CI fallido

El cookbook oficial documenta un workflow de GitHub Actions que corrige automáticamente tests fallidos: detecta failure → checkout → codex exec → verify → create PR.

```
name: Codex auto-fix on CI failure
on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]

jobs:
  auto-fix:
    if: ${ github.event.workflow_run.conclusion == 'failure' }
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: openai/codex-action@v1
        with:
          openai-api-key: ${ secrets.OPENAI_API_KEY }
          prompt: |
            Read the repository, run the test suite,
            identify the minimal change to pass all tests.
            Do not refactor unrelated files.
          sandbox: workspace-write
      - run: npm test --silent
      - uses: peter-evans/create-pull-request@v6
        if: success()
```

3.4 codex exec resume — Pipelines multi-paso

Para pipelines de dos o más etapas, codex exec resume continúa una sesión anterior con todo el contexto previo:

```
# Paso 1: review
codex exec "review the change for race conditions"

# Paso 2: fix basado en el review
codex exec resume --last \
  "fix the race conditions you found"
```

3.5 Codex Cloud

Codex Cloud ejecuta tareas en contenedores remotos con el repo clonado desde GitHub. Environments con setup scripts, variables y secrets encriptados. Cache de contenedor hasta 12 horas. Internet del agente desactivado por default.

```
# Desde CLI
codex cloud exec --env ENV_ID "Fix the failing tests"

# Best-of-N (1-4 intentos)
codex cloud exec --env ENV_ID --attempts 3 "Summarize bugs"
```

3.6 Worktrees en la App

La Codex App usa Git worktrees para aislar cambios: cada worktree tiene copia propia de ficheros pero comparte metadata .git. Permite trabajar en paralelo sin romper el checkout principal. Limpieza automática al archivar el thread.

3.7 Buenas prácticas Git con Codex (oficial)

La documentación de seguridad de Codex recomienda:

- Feature branch: trabajar en rama, git status limpio antes de delegar.
- Patch-based workflows: preferir git diff/git apply sobre editar tracked files.
- Commits frecuentes: rollback en incrementos pequeños.
- Tratar sugerencias como PRs: verificar diffs, ejecutar tests, documentar en commits.

3.8 Configuración para equipos

3.8.1 requirements.toml (Enterprise)

```
allowed_approval_policies = ["on-request", "untrusted"]
allowed_sandbox_modes = ["read-only", "workspace-write"]
allowed_web_search = ["cached"]

[[mcp_servers]]
id = "openaiDocs"
url = "https://developers.openai.com/mcp"
```

3.8.2 Perfiles compartidos

```
# .codex/config.toml (proyecto trusted)
[profiles.ci]
sandbox_mode = "workspace-write"
approval_policy = "never"
model_reasoning_effort = "medium"

[profiles.review-only]
sandbox_mode = "read-only"
approval_policy = "untrusted"
```

3.8.3 Execution policy rules

```
# .codex/rules/team.rules
prefix_rule("rm -rf /", "forbidden", "Destructivo")
```

```
prefix_rule("git push --force", "prompt", "Force push")
prefix_rule("git rebase", "prompt", "Rebase")
prefix_rule("pytest", "allow", "Tests permitidos")
```

3.8.4 OTel para auditoría

```
[otel]
environment = "prod"
exporter = "otlp-http"
log_user_prompt = false

[otel.otlp_http]
endpoint = "https://otel.internal.company.com"
```


4. Buenas prácticas

4.1 Codex propone, humano aprueba

Nunca automatizar merges sin revisión humana. `approval_policy = "on-request"` como default de equipo.

4.2 PRs pequeños

Cada tarea de Codex = un PR. No mezclar múltiples features.

4.3 CI como árbitro

"No merge sin verde." Branch protection rules en GitHub.

4.4 Commits convencionales

Pedir formato `type(scope): description` (feat, fix, refactor, test, docs, chore).

4.5 Execution rules para proteger

forbidden en `rm -rf /`, prompt en `git push --force` y `git rebase`.

4.6 Límites de auto-fix

Máximo 3 intentos de codex exec en CI. Si no resuelve, escalar a humano.

4.7 Secrets solo en setup

En Codex Cloud, los secrets se borran antes de la fase del agente.

5. Errores comunes

5.1 Reescribir historial sin policy

Problema: git push --force o git rebase sin aprobación.

Solución: Execution rule con prompt para force push y rebase.

5.2 Automatizar merges sin revisión

Problema: CI verde no significa que el cambio es correcto.

Solución: Branch protection + revisión humana requerida.

5.3 danger-full-access en CI compartido

Problema: Acceso a secrets de otros jobs.

Solución: drop-sudo (default) + workspace-write.

5.4 No pinear versión de Codex

Problema: Updates inesperados rompen pipelines.

Solución: codex-version en GitHub Action.

5.5 Prompts de CI sin constraints

Problema: "Fix all bugs" es demasiado amplio.

Solución: "Fix ONLY the failing test. Do not refactor unrelated files."

6. Casos de uso reales

6.1 Migración gradual

PRs por módulos con Codex Cloud. Cada PR incluye cambios + tests + verificación. Rollback simple: revert del PR.

6.2 Release notes automáticas

`codex exec "generate release notes for the last 10 commits"` en pipeline de release.

6.3 Changelogs estructurados

`codex exec con --output-schema` para generar changelogs en formato JSON consumible por herramientas downstream.

6.4 Triage de issues

@codex en issues de GitHub para que analice y proponga fixes como PRs desde Codex Cloud.

7. Laboratorio práctico (L10) — PR Listo para Producción

🎯 Objetivo del laboratorio

Crear un repo con pipeline CI simulado, usar Codex para generar cambios + tests + PR description, simular auto-fix de CI fallido, y verificar que todo funciona end-to-end.

7.1 Paso 1 — Crear proyecto con CI simulado

```
mkdir /tmp/codex-lab10 && cd /tmp/codex-lab10
git init

python3 -m venv .venv
source .venv/bin/activate
pip install pytest flake8

mkdir -p src tests .github/workflows

# Crear calculadora base + tests + ci.sh
# (ver markdown para código completo)

chmod +x ci.sh
PYTHONPATH=. pytest tests/ -v
git add -A && git commit -m "chore: proyecto base con CI"
```

7.2 Paso 2 — AGENTS.md y config de equipo

```
# AGENTS.md con convenciones Git y testing
# .codex/config.toml con perfiles ci-auto y review-only
# .codex/rules/team.rules con execution policy
git add -A && git commit -m "chore: AGENTS.md + config"
```

7.3 Paso 3 — Feature branch + cambios con Codex

```
git checkout -b feat/multiply-divide
codex --full-auto

# Prompt: añadir multiply/divide con tests
# Codex genera, ejecuta ci.sh, commitea con
# formato feat(calculator): ...
```

7.4 Paso 4 — PR description con codex exec

```
codex exec \
  "Genera PR description en Markdown:
  What, Why, How to test, Checklist" \
  -o pr-description.md
```

```
cat pr-description.md
```

7.5 Paso 5 — /review antes de merge

```
codex
# En TUI: /review → Review against base → main
```

7.6 Paso 6 — Simular auto-fix con codex exec

```
# Introducir bug deliberado en power()
# ci.sh falla (esperado)

codex exec --full-auto \
  "Run ./ci.sh. If tests fail, identify minimal
  change to pass. Do not refactor unrelated files.
  Run ./ci.sh again to verify."

./ci.sh # Debe pasar ahora
```

7.7 Verificación final

Verificación	Resultado esperado
CI pasa	./ci.sh → CI PASSED
Tests cubren edge cases	multiply×0, divide/0, power^0
Coverage >= 80%	--cov-fail-under=80 pasa
Commits convencionales	feat(...), test(...), fix(...)
PR description generada	What + Why + How to test + Checklist
/review sin P0/P1	Hallazgos graves resueltos
Auto-fix funcionó	codex exec corrigió bug de power
Execution rules activas	git push --force requiere prompt

7.8 Limpieza (Protocolo Obligatorio)

⚠ Limpieza de recursos
Ejecutar al finalizar:

```
deactivate
rm -rf /tmp/codex-lab10

ls /tmp/codex-lab10 2>/dev/null \
  && echo 'ERROR: aún existe' \
```

```
|| echo 'OK: limpio'
```

8. Resumen — Cierre del Plan de Formación

Este módulo concluye el plan de formación de 10 módulos sobre OpenAI Codex. A lo largo del plan hemos cubierto:

Módulo	Tema clave
M1	Fundamentos: qué es Codex, arquitectura, primer uso
M2	Instalación y autenticación
M3	Selección de modelos y reasoning effort
M4	Gestión de contexto: @files, /mention, /compact, AGENTS.md
M5	Skills: diseño de instrucciones reutilizables
M6	Configuración profesional: sandbox, approval, rules, perfiles
M7	MCP: servidores STDIO/HTTP, tools externas
M8	Refactorización, /review, optimización con baselines
M9	Testing y debugging: pirámide de tests, auto-fix CI
M10	CI/CD: codex exec, GitHub Action, Cloud, team config

🏠 Formación completada

El alumno que ha completado los 10 módulos puede: usar Codex CLI, IDE y Cloud profesionalmente; integrar Codex en pipelines CI/CD; configurar para equipos; y auditar con OTel. Los laboratorios prácticos proporcionan experiencia reproducible con cada funcionalidad.

9. Referencias

- Non-interactive (codex exec): <https://developers.openai.com/codex/noninteractive/>
- GitHub Action: <https://developers.openai.com/codex/github-action/>
- Auto-fix CI cookbook: <https://developers.openai.com/cookbook/examples/codex/autofix-github-actions/>
- SAST GitLab cookbook: https://developers.openai.com/cookbook/examples/codex/secure_quality_gitlab/
- Security: <https://developers.openai.com/codex/security/>
- Codex Cloud: <https://developers.openai.com/codex/cloud/>
- Cloud environments: <https://developers.openai.com/codex/cloud/environments/>
- Worktrees: <https://developers.openai.com/codex/app/worktrees/>
- Advanced config: <https://developers.openai.com/codex/config-advanced/>
- Team config: <https://developers.openai.com/codex/config-advanced/> (Team Config section)